

Through WebAssembly (browser)

Impulses can be deployed as a WebAssembly library. This packages all your signal processing blocks, configuration and learning blocks up into a single package. You can include this package in web pages, or as part of your Node.js application. This allows you to run your impulse locally, without any compilation. In this tutorial you'll export an impulse, and build a web application to classify sensor data. You can also load this library from Node.js, see [Through WebAssembly \(Node.js\)](#).

Prerequisites

Make sure you followed the [Continuous motion recognition](#) tutorial, and have a trained impulse. Also install the following software:

- Python 3 - to run a web server that serves the MIME type of the `.wasm` file correctly.

Deploying your impulse

Head over to your Edge Impulse project, and go to **Deployment**. From here you can create the full library which contains the impulse and all external required libraries. Select **WebAssembly** and then click **Build** to create the library. Download and unzip the `.zip` file.

Then, create a new file called `run-impulse.js` in the same folder, and add:

`run-impulse.js`

```
return new Promise((resolve) => {
  Module.onRuntimeInitialized = () => {
    resolve();
    classifierInitialized = true;
  };
});
}

classify(rawData, debug = false) {
  if (!classifierInitialized) throw new Error('Module is not initialized');

  const obj = this._arrayToHeap(rawData);
  let ret = Module.run_classifier(obj.buffer.byteOffset, rawData.length, debug);

  Module._free(obj.ptr);

  if (ret.result !== 0) {
    throw new Error('Classification failed (err code: ' + ret.result + ')');
  }

  let jsResult = {
    anomaly: ret.anomaly,
    results: []
  };
};
```

```

        c.delete();
    }
}
else {
    for (let cx = 0; cx < ret.classification.size(); cx++) {
        let c = ret.classification.get(cx);
        jsResult.results.push({ label: c.label, value: c.value });
        c.delete();
    }
}

ret.delete();

return jsResult;
}

_arrayToHeap(data) {
    let typedArray = new Float32Array(data);
    let numBytes = typedArray.length * typedArray.BYTES_PER_ELEMENT;
    let ptr = Module._malloc(numBytes);
    let heapBytes = new Uint8Array(Module.HEAPU8.buffer, ptr, numBytes);
    heapBytes.set(new Uint8Array(typedArray.buffer));
    return { ptr: ptr, buffer: heapBytes };
}
}

```

Also, add a file called `server.py` and add:

`server.py`

```

import http.server
from http.server import HTTPServer, BaseHTTPRequestHandler
import socketserver

```

```

PORT = 8082

```

```

Handler = http.server.SimpleHTTPRequestHandler

```

```

Handler.extensions_map={
    '.manifest': 'text/cache-manifest',
    '.html': 'text/html',
    '.png': 'image/png',
    '.jpg': 'image/jpeg',
    '.svg': 'image/svg+xml',
    '.css': 'text/css',
    '.js': 'application/javascript',
    '.wasm': 'application/wasm',
    '': 'application/octet-stream', # Default
}

```

```

httpd = socketserver.TCPServer(("", PORT), Handler)

```

```

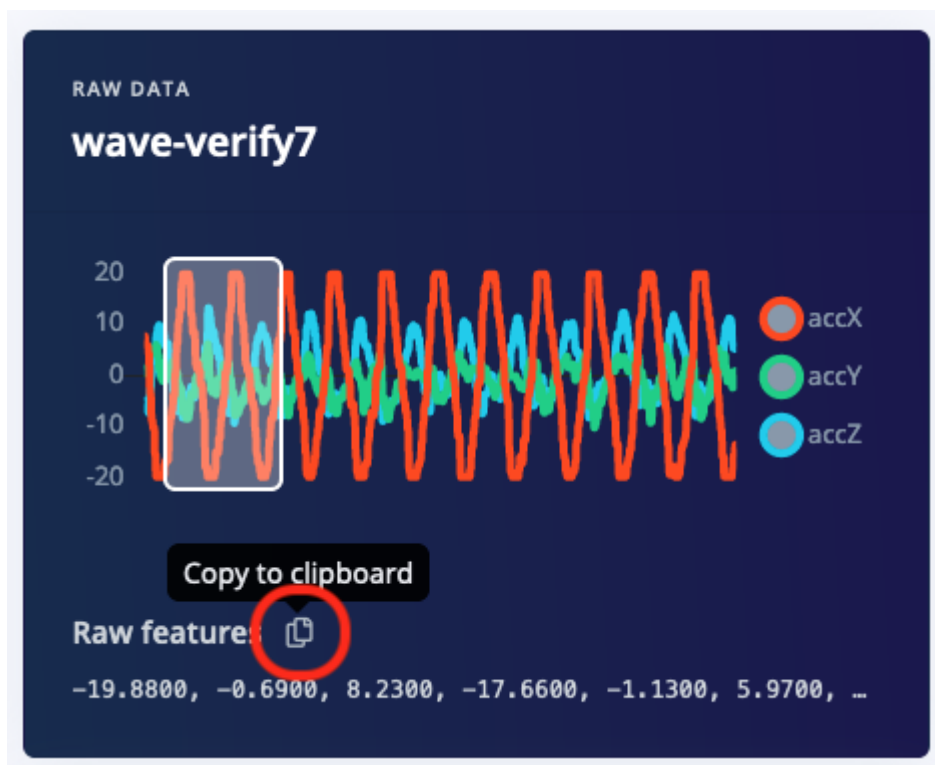
print("serving at port", PORT)
httpd.serve_forever()

```

TIMESTAMP	IDLE	SNAKE	UPDOWN	WAVE	ANOMALY
0	0	0	0	0.99	0.42
80	0.04	0	0.01	0.95	0.35
160	0	0	0	0.99	0.25
240	0	0	0	0.99	0.19
320	0.02	0	0	0.98	0.13
400	0	0	0	0.99	0.09

Selecting the row with timestamp '320' under 'Detailed result'.

To verify that the local application classifies the same, we need the raw features for this timestamp. To do so click on the 'Copy to clipboard' button next to 'Raw features'. This will copy the raw values from this validation file, before any signal processing or inferencing happened.



Copying the raw features

Then, create a new HTML file where you'll call the inferencing engine with these features. Add a new file called `index.html` and add:

```
index.html
```

```
</head>
<body>
  <script src="edge-impulse-standalone.js"></script>
  <script src="run-impulse.js"></script>
  <script>
    (async () => {
      var classifier = new EdgeImpulseClassifier();
      await classifier.init();
      console.log('results', classifier.classify([
        // YOUR FEATURES HERE
        -19.8800, -0.6900, 8.2300, -17.6600, -1.1300, 5.9700, ...
      ]));
    })();
  </script>
</body>
</html>
```

Make sure to replace the features under `YOUR FEATURES HERE`.

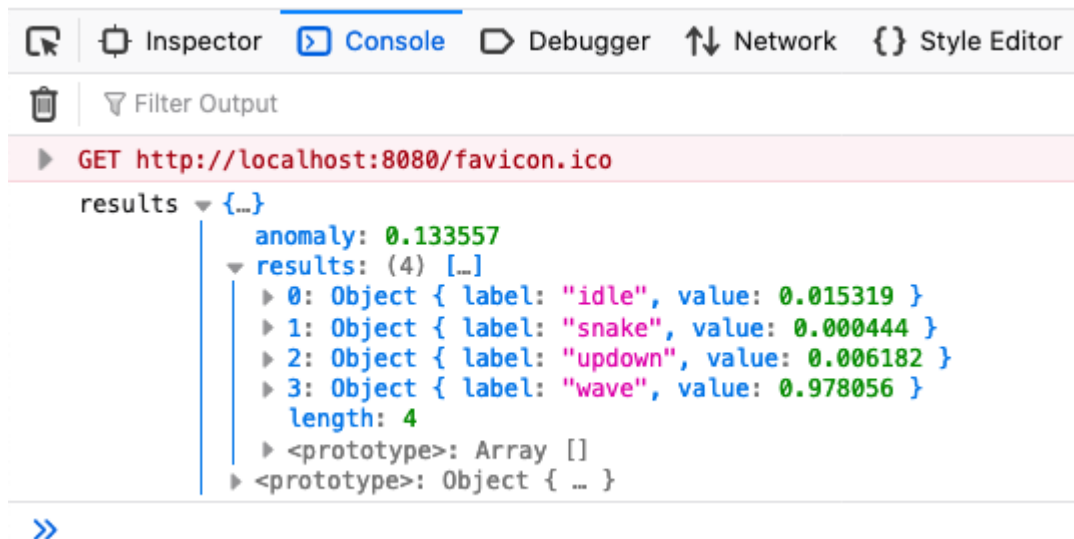
Then run the application via:

```
$ python3 server.py
```

And navigate to <http://localhost:8082> to see the application.

Seeing the output

To see the output of the impulse, open the developer console in your browser.



Seeing the results of your impulse running in the browser.

Which matches the values we just saw in the studio. You now have your impulse running in your browser!